

```

        theText, width=WINDOW_WIDTH,
        justified='center', fontSize=36)
    ❸ oNoButton = pygwidgets.CustomButton(theWindow, (95, 265),
        'images/noNormal.png',
        over='images/noOver.png',
        down='images/noDown.png',
        disabled='images/noDisabled.png')
    oYesButton = pygwidgets.CustomButton(theWindow, (355, 265),
        'images/yesNormal.png',
        over='images/yesOver.png',
        down='images/yesDown.png',
        disabled='images/yesDisabled.png')
    ❹ userAnswer = pyghelpers.customYesNoDialog(theWindow,
        oDialogBackground,
        oPromptDisplayText,
        oYesButton, oNoButton)

    ❺ return userAnswer

```

Listing 16-1: An intermediate function to create a custom Yes/No dialog

Inside the function you write code to create an `Image` object for the background using an image you specify ❶. You also create a `DisplayText` object for the prompt ❷, in which you specify the placement, text size, font, and so on. Then you create buttons as either `TextButton` objects or, more likely, `CustomButton` objects so you can show custom images ❸. Finally, this function calls `customYesNoDialog()`, passing in all the objects you just created ❹. The call to `customYesNoDialog()` returns the user's choice to this intermediate function, and the intermediate function returns the user's choice to the original caller ❺. This approach works well because the widget objects (`oDialogBackground`, `oPromptDisplayText`, `oYesButton`, and `oNoButton`) created inside this function are all local variables and therefore will all go away when the intermediate function ends.

When you call this function, you only need to pass in the window and the text prompt to be displayed. For example:

```

returnedValue = showCustomYesNoDialog(window,
                                     'Do you want fries with that?')

```
